2021 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW) | 978-1-6654-4456-9/20/$31.00 ©2021 IEEE | DOI: 10.1109/ICSTW52544.2021.00020

A Practical Method for API Testing in the Context of Continuous Delivery and Behavior Driven Development

Brian Elgaard Bennett
OpenAPI Product Team
Saxo Bank A/S
Hellerup, Denmark
bb@saxobank.com

***Abstract*—Enterprises are increasingly adopting an API-first approach to connect and expose software services. Saxo Bank is no exception to this.**

Crafting test suites for such APIs can seem straight forward due to the headless nature, but our experience shows that test suites often have two problems. The first problem is that execution of tests tends to fail and pass in seemingly nondeterministic ways (tests are *flaky*). The second problem is that functional coverage is not clearly documented.

We have found that both problems stem from a lack of explicit focus on *initial context (IC)*, a concept from behavior driven development. When a test is flaky it is often because actual IC in the test environment is not as required by the test. When functional coverage is not clear, it is most often because a systematic analysis involving IC was not performed.

We propose a method for test analysis in which we *include IC* in the input space when analyzing functional coverage for an API, thereby including anything which can influence the outcome of test cases.

Establishing IC is in general a hard problem. We have found that focus on the *bounded context*, a concept from domain driven design, of the system under test is a practical way to establish relevant IC.

Experience with Saxo Bank's Open API shows that this method allows testers and developers to cooperate continuously, producing test plan documents which include the reasoning behind functional coverage. Explicit focus on IC in automated test case implementations turns flaky tests into tests which report on required IC in a test environment. The method easily generalizes to all levels of API tests.

Keywords—Combinatorial Testing, API Testing, Behavior Driven Development, Domain Driven Design, Continuous Delivery, Industry, Application

I. Introduction and Background

The first version of Saxo Bank's Open API was largely tested with unit tests written by developers, and manual tests executed by testers. Unfortunately, the result of an effort to achieve better quality and faster releases by automating the manual tests was less than perfect. The automated tests failed and passed in seemingly nondeterministic ways - the tests were *flaky*. Also, there was no systematic way to document intended and achieved functional coverage, i.e. there was no way to determine what was tested and what was not tested from a functional point of view. So even a successful test run did not provide clear confidence for relevant stakeholders.

We turned to behavior driven development (BDD) for guidance on addressing these two problems. BDD can be seen as an evolution of test driven development (TDD), the difference of the two being that BDD tends to focus on high-level exposed behavior, whereas TDD tends to focus on low-level implementation details.

The de facto standard for writing BDD tests is to use the Gherkin language, which uses keywords *Given*, *When* and *Then*. Here is an example of a simple test case,

Given I have a user John Doe with ID 42
When I GET user 42
Then the John Doe user should be returned

We see the Given clause above as a declaration of the test case's assumption on *initial context (IC)* in the execution environment. It is in general a hard problem to establish IC for test suites, but we find that this problem is manageable when we specify IC *per test case*. The mechanism to establish IC in the execution environment could be a full baseline from a production database or it could be synthetic data, possibly containing but the single user object assumed by this single test case.

Focus on IC per test case provided us with a way to handle flaky tests. We delegated handling of assumptions to a common test infrastructure service so that tests could report when assumptions in Given clauses could not be fulfilled. In such a case, a test will not fail, rather it will report that it cannot be executed, and it reports the reason for this in terms of specific IC.

It was tempting to believe that this was also a solution to the second problem, to determine what was covered by test cases from a functional point of view. After all, reading through the set of Gherkin files is all it takes to determine what is covered by test cases. However, it was our experience that it is difficult to get an overview based on a large host of Gherkin files, and without a systematic way to link requirements to test cases, it is equally difficult to determine what parts of requirements are covered by the test cases.

978-1-6654-4456-9/21/$31.00 ©2021 IEEE
DOI 10.1109/ICSTW52544.2021.00020

Authorized licensed use limited to: University of Prince Edward Island. Downloaded on July 03,2021 at 17:52:30 UTC from IEEE Xplore. Restrictions apply.

We turned to combinatorial testing (CT) to introduce systematism. Our idea was to model the input space using not only input parameters but also IC. The immediate problem with our idea was the high number of possible combinations. CT gives a (partial) solution to this since empirical data show that the number of variables involved in failures is small [1], so tests for t-way combinations will likely cover a large percentage of failures, even for low values of t. While we realized that it was unrealistic to have complete coverage of possible failures, our ambition became to document coverage in terms of functional requirements rather than in terms of probabilities from empirical studies. We knew about potential failures in our business domain for which at least *t=6* would be needed, so choosing a "high enough *t*" for entire test suites would yield an extreme number of tests and this approach would not be feasible.

We then tried to tame the combinatorial explosion by using domain knowledge to identify areas of functionality. This worked well for us and we will illustrate this in the rest of this paper.

II. The System Under Test – Bounded Context

A. Description of the Architecture

Saxo Bank's Open API consists of an increasing number of services which expose functionality within the domain of financial instruments trading. Each service typically exposes functionality of trading the financial markets through REST or gRPC endpoints. The architecture is highly complex, in the following we will restrict ourselves to a representative simplification.

Each individual service exposes a set of APIs related to a sub-domain, as illustrated in Fig. 1.

The service (S0) in Fig. 1 has three dependencies, two other services (S1 and S2) and a database (DB0). If we were to *test S0 in isolation*, the input space of S0 would, in addition to input parameters to API0, also include IC from S1, S2 and DB0. This would be typical for in-memory tests (unit tests) in which the three dependencies would be replaced by test doubles.

Now we “zoom out” one level while still targeting S0 and API0, as illustrated in Fig. 2. If we were to *test these three services in isolation*, the input space would again include input parameters to API0 and IC from DB0. But this time, we include IC from the dependencies' dependencies. In other words, we include IC from the services S11 and S21 and the databases DB1

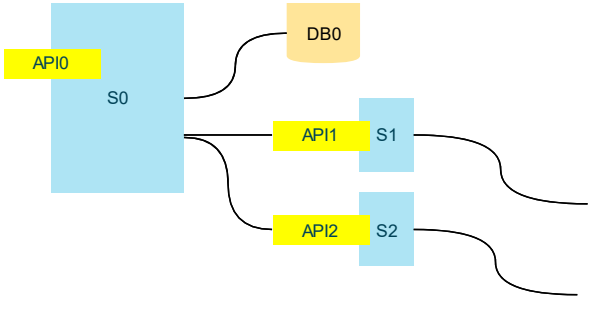


Fig. 1. Service S0 exposes functionality through API0 and depends on services S1 and S2.

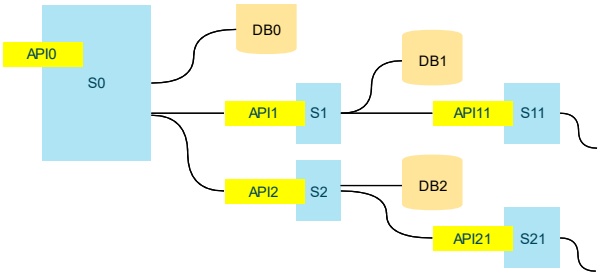


Fig. 2. Service S0's dependencies' dependencies

and DB2. In this example, it is assumed that services exclusively keep state in their respective databases.

This is typical for out-of-memory tests which integrate several services in a test environment. In this case, the system under test (SUT) is the three services and three databases. S11 and S21 are not part of the SUT and will only be considered for integration testing.

It is important to define such tangible boundaries of sub-domains for a continuous delivery (CD) process. Inspired by *domain driven design (DDD)*, we call a sub-domain within such boundaries a *bounded context (BC)*. In an ideal architecture, each BC can be developed and tested in isolation and deployed to production independently of other parts of the system.

Experience with our proposed method for test analysis shows that focus on a well-defined BC with equally well-defined IC tend to produce test suites which accurately cover the intended SUT. Often, this focus helps participants in the process to drastically simplify the input space.

B. Levels of Tests

It is common to depict levels of tests using the metaphor "test pyramid", illustrating that it is desired to have more of the tests at the base, less in the middle and even less at the top. Traditionally for API tests, in-memory tests (unit tests) are at the base, integration tests are in the middle and end-to-end (E2E) tests are at the top.

There are huge practical differences between in-memory tests and out-of-memory tests. However, in this paper, we will ignore these practicalities because our focus, IC, is conceptually the same in both cases. Our handling of IC in in-memory tests follows the principle of Inversion of Control (IoC), using dependency injection (DI) containers. We also conceptually handle IC in out-of-memory tests using IoC, but we use a combination of cloud technologies rather than DI containers. The details are out of the scope of this paper.

For Saxo Bank's legacy services, we need to do E2E testing in shared test environments, but for new services the highest level of tests is integration tests, which cover integration to the dependencies of a BC. In a shared environment, it is generally challenging to get fully deterministic outcomes of test runs, since IC can only be *assumed*, and to a certain extent *verified*, but IC can generally not be *ensured*.

Again, for the purpose of this paper, we will ignore these complexities. Suffice it to note that explicit focus on IC per test allows each test to indicate the specific IC which is required for the test to run.

Authorized licensed use limited to: University of Prince Edward Island. Downloaded on July 03,2021 at 17:52:30 UTC from IEEE Xplore. Restrictions apply.

III. The Method for Test Analysis

A. A Complex Real-World Example - an Order Endpoint

Functionality for placing an order is core to Saxo Bank's Open API. The *Trading service* exposes this functionality as a REST *POST* endpoint.

Parameters include a financial *asset* identified by an *id* and an *asset type*. Other parameters indicate if the order is *buy* or *sell*, the requested *amount*, the *duration* for the order to stay in the market etc. The *order type* is an example of a parameter which potentially can interact with other parameters. The *market* entry order type is simple in the sense that it does not even interact with a *price* parameter. In contrast to this, a *trailing stop* related order type interacts not only with a price parameter, but also with *distance* and step *parameters*.

A relevant IC item is *the logged-in user*. This IC item includes authorization and authentication information, e.g. if the user has permission to trade the asset type or if the user is even allowed to execute the order endpoint. For a sell order, it can indicate if the logged-in user owns the asset to sell. Owning the asset would be required for the asset type *stock* but is not necessarily required for the asset type *contract for difference (CFD)*, since selling a CFD which you do not own means *going short*. Then again, the user may not be allowed to trade CFDs.

So far, we have only mentioned IC items related to the logged-in user. Other relevant IC items include information on assets. A logged-in user may be allowed to trade assets of type stock, but if the particular stock indicated by an id parameter does not exist in an *instruments repository*, or if the stock is delisted, it would be a failure not to return an error.

It is not important in the following to fully understand ordering functionality in general or the above-mentioned functional requirements. Rather, it is important to understand that this single endpoint is complex with respect to the business domain, parameters and IC items, and that achieving sufficient confidence through testing is a non-trivial task.

B. Covering the Order Endpoint with Tests

The above description of the order endpoint only touches upon a small part of its complexity. There are 34 parameters in the actual order endpoint, some of the parameters are composite and thus hide an even larger number of parameters. Some parameters have many relevant values, e.g. there are 58 different asset types. The number of relevant IC items is also vast.

We could try to do t-way CT and hope to cover the most important failures. However, due to the large number of parameters and IC items, only a low t would be feasible. Alas, we know at least one important case which appears to require at least t=6, namely placing a *trailing stop related order*, for which input parameters must indicate at least *asset type*, *amount*, *price*, *distance* and *step*, while IC items must include *tradability* of the asset and possibly information on *the logged-in user's ownership of the asset*.

C. A Simplified Example - an Order Endpoint

In this simple example we have cut down the full example to fit this paper. We have restricted the number of parameters and IC items to 10 and 2, respectively. We have restricted the number of values of each of *asset type* and *order type* to 2.

Input parameters and possible values are, *asset type* (*stock*, *CFD*), *order type* (*limit*, *trailing step*), *buy/sell type* (*buy*, *sell*), *order duration* (a date with time), *id* (an integer), *price* (a decimal number), *amount* (a decimal number), *position id* (an integer), *distance* (a decimal number) and *step* (a decimal number). IC items are *logged-in user* and *asset*.

In the following we will emphasize that we treat IC items on par with input parameters, by referring to both using the common term *dimensions*.

D. The Analysis Steps

The first step is to *group dimensions* according to how they are known to interact within *areas of functionality*. Our experience shows that equivalence partitioning and boundary analysis will most often yield significantly fewer possible values for each dimension of an area than for the same dimension for the whole. We use domain knowledge to do this grouping, but we do not restrict ourselves to black box testing, as equivalence partitioning and boundary analysis is often further helped by inspecting code paths. This is possible due to close and continuous cooperation between tester and developer.

More importantly, the second step is to carefully *consider the BC* which represents our SUT. In our example, the BC is the Trading service itself, as illustrated in Fig. 3.

Such consideration will often reveal that required functionality is implemented outside our SUT and is thus not relevant to cover, except for input validation and integration to direct dependencies. This step is surprisingly often overlooked, the consequence being that the testing effort is often not sufficiently targeting the relevant BC.

Considering the BC is also key to identifying IC items. For example, the asset being traded above is a complex data structure, spanning multiple database tables, and with complex associated business logic. However, since such functionality and data structures are handled outside the BC under test, from the point of view of the BC, we can define abstract items such as e.g. a *tradable CFD*. With this simplification, we can ignore the fact that complex functionality and data structures are needed to determine whether a given CFD is tradable or not. This means that IC items can be handled in an analysis similar to how input parameters are handled.

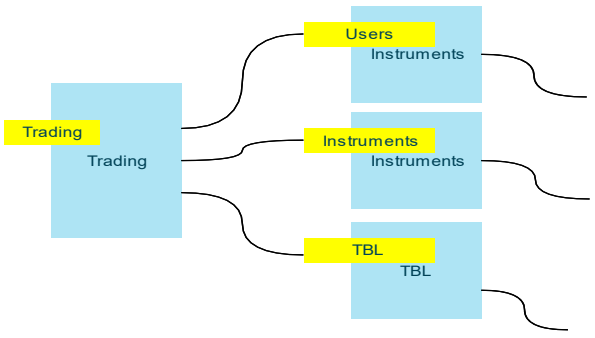


Fig. 3 The BC of the Trading service is the service itself.

Authorized licensed use limited to: University of Prince Edward Island. Downloaded on July 03,2021 at 17:52:30 UTC from IEEE Xplore. Restrictions apply.

First step, group dimensions—Based on requirements, it seems that interactions vary significantly for different *order types*. We therefore decide to group by order type, so that we have one group for *limit* entry orders and another group for *trailing step* related orders.

Second step, consider the BC—Requirements indicate interaction among several dimensions, as e.g. it may be allowed to sell an asset that the logged-in user does not own, but this depends on at least the logged-in user, the asset type and a concrete asset IC item. However, by closer inspection of the requirements, architectural documentation, and actual code paths, it can be concluded that this functionality is entirely implemented outside the BC and is thus not relevant to cover. It can also be concluded that the two asset types can be collapsed into a single equivalence class.

Within the BC, the responsibilities of the Trading service are (1) advanced input parameter validation, and (2) mapping combinations of input parameters to the API of the dependent *TBL* service (see Fig. 3), which will then do the actual order placement.

Armed with this knowledge, we can conclude that some of the dimensions don't contribute to the combinatorial explosion for a given group (order type), as we can use any asset type and we can assume that the user is authenticated and that authorization to trade the asset is handled outside the BC. Therefore, we will omit these dimensions from further analysis.

Next, we *consider 1-way interactions*. This is typically simple input validations. In our example, this is relevant for the id, price, amount, and order duration parameters. Covering a single invalid value for each is sufficient, so 4 tests will cover these (see TABLE *1*). In the following analysis, we can use any valid value for these dimensions. We will restrict ourselves to analysis of the trailing step related order functionality.

TABLE 1. 1-WAY INTERACTIONS

Id	**Price**	**Amount**	**Duration**	**Other dims**	**Test case**
Invalid	Valid	Valid	Valid	Valid	InvalidId
Valid	Invalid	Valid	Valid	Valid	InvalidPrice
Valid	Valid	Invalid	Valid	Valid	InvalidAmount
Valid	Valid	Valid	Invalid	Valid	InvalidDuration

Trailing step related order test analysis—We now have the following dimensions and values to consider, *buy/sell type* (*buy*, *sell*), *position id* (*valid*, *invalid*, *not there*), *distance* (*valid*, *invalid*, *not there*), *step* (*valid*, *invalid*, *not there*) and *asset* (*tradable*, *not tradable*).

We are down to only 108 combinations, which is manageable, especially if we assume only 2-way interaction, in which case we can reduce the number to 11. But this is but a simplified example, and in general we want to deduce actual interaction, so we continue one step further based on our knowledge about Trading service responsibilities.

The parameters *position ID*, *distance* and *step* are handled by *advanced input parameter validation*. However, looking at requirements and code paths, it is reasonable to assume that validation functionality of the three parameters do not interact. So again, we need 3 tests for invalid values and 3 tests for the values not provided (not shown in a table). After this, we can assume that these three parameters do not contribute to the combinatorial explosion.

The only parameters we still need to consider are the *buy/sell type* and the *asset being tradable or not*, in total 4 tests (see TABLE *2*).

TABLE 2. BUY/SELL AND ASSET TRADABILITY INTERACTIONS

Buy/sell	**Asset**	**Test case**
Buy	Tradable	BuyTradable
Sell	Tradable	SellTradable
Buy	Not tradable	BuyNotTradable
Sell	Not tradable	SellNotTradable

In conclusion, the analysis has shown that we can cover the required functionality with 4+3+3+4 = 14 tests.

More importantly, the outcome of the analysis is a *test plan document* which documents our reasoning. The test plan is saved under version control along with the source for production code and automated test cases. The test plan *references* requirements and *discusses and reasons* about requirements. It is our experience that the analysis process itself provides important value, as humans involved gather important understanding and deliver well-argued and focused test cases.

The Test case column in the tables shown allows our custom tooling to link test analysis to actual test case implementations. This allows for early feedback on the correlation between test analysis and actual test runs in CI/CD pipelines. This is an important benefit of our method, but tooling is not in the scope of this short paper.

We have not discussed oracles so far. In short, we craft oracles manually per test case based on observation of the difference between IC and the *resulting state*. Oracles are easy to correlate to test analysis due to the linking mentioned above.

CONCLUSIONS

Focus on IC per test case has allowed us to solve problems with API testing which so far had seemed unsolvable.

For integration and end-to-end tests we have turned most test cases which failed or passed seemingly at random into test cases which either fail or pass as expected or clearly indicate what is required for the test case to run.

For low-level, in-memory test cases, it is trivial to achieve full determinism, but focus on IC per test case, even at this level, allows us to be conceptually and syntactically consistent across all levels of test.

Inspired by CT, BDD and DDD we have used this consistency to propose a method for documenting reasoning as part of test analysis.

REFERENCES

[1] D.R. Kuhn, R.N. Kacker, and Y. Lei, "Introduction to Combinatorial Testing," Chapman & Hall/CRC Innovations in Software Engineering and Software Development Series, Taylor & Francis, 2013.

Authorized licensed use limited to: University of Prince Edward Island. Downloaded on July 03,2021 at 17:52:30 UTC from IEEE Xplore. Restrictions apply.